Versi 1.2

September 2026

STRUKTUR DATA

MODUL 2 - ARRAY LIST & LINKED LIST

Author:

FERY PRATAMA PUTRA

MUHAMMAD IBRAHIM AL AYUBI

MARENO ALI IBRAHIM

INFORMATICS LABORATORY

University of Muhammadiyah Malang

DAFTAR ISI

DAFTAR ISI **2**
INTRODUCTION **4**
TUJUAN 4
TUJUAN MODUL 4
PERSIAPAN 4
KATA KUNCI 4
ISI **5**
1. Array List **5**
1.1. Definisi ArrayList 5
1) ArrayList() 5
2) ArrayList(Collection<? extends E> c) 5
3) ArrayList(int initialCapacity) 5
1.2. Karakteristik ArrayList 6
1.3. Cara ArrayList Bekerja Secara Internal 6
1.4. Metode ArrayList Penting 8
1.5. Contoh Kasus Penggunaan ArrayList 9
1) Deklarasi ArrayList 9
2) add(element) 10
3) add(index, element) 10
4) get(index) 10
5) set(index,element) 11
6) remove(index) 11
7) size() 11
8) isEmpty() 12
9) clear() 12
2. Linked List **12**
2.1. Definisi LinkedList 12
2.2. Struktur Dasar LinkedList 12
1) Node 13
2) Data 13
3) Next & Previous Point 13
4) Pointer 14
6) Tail 15
2.3. Jenis-Jenis LinkedList 15
1) Singly Linked List 15
2) Doubly Linked List 16
3) Singly Circular Linked List 16
4) Doubly Circular Linked List 16
2.4. Operasi LinkedList Penting 17

1) Insertion..... 17
2) Deletion.....17
3) Traversal..... 18
4) Searching.....18
5) Updating..... 18
6) Menghitung Node..... 18
2.5. Contoh Kasus Penggunaan LinkedList.....18
CODELAB..... 26
DEMO..... 28
TASK 1 Passenger Management System (ArrayList dan LinkedList).....28
Grading Rubric.....30
Grading Scale..... 30

INTRODUCTION

TUJUAN

1. Memahami konsep dasar ArrayList dan LinkedList, serta perbedaan struktur internalnya.
2. Menjelaskan karakteristik ArrayList dan LinkedList.
3. Mengimplementasikan ArrayList dan LinkedList di Java secara mandiri.
4. Menggunakan metode penting seperti add, remove, get, set, traversal, search, update, dan count.
5. Menganalisis dan memilih struktur data yang sesuai berdasarkan kebutuhan studi kasus.
6. Membuat simulasi manajemen data menggunakan ArrayList dan LinkedList (misalnya sistem pemesanan tiket, antrean penumpang, dan daftar dinamis).
7. Membangun pemahaman visual tentang bagaimana node, pointer, head, tail, dan hubungan antarnode bekerja.

TUJUAN MODUL

1. Menjelaskan konsep serta perbedaan antara ArrayList dan LinkedList sebagai struktur data dinamis di Java.
2. Mengidentifikasi karakteristik, kekuatan, dan kelemahan ArrayList dan LinkedList untuk berbagai kasus penggunaan.
3. Mengimplementasikan ArrayList dan LinkedList dalam program Java menggunakan sintaks dan konstruktor yang benar.
4. Gunakan metode penting (add, remove, get, set, size, clear, traversal, search, update) secara efektif pada kedua struktur data tersebut.
5. Memvisualisasikan dan menjelaskan bagaimana node, pointer, head, tail, dan link bekerja dalam linked list.
6. Mengembangkan aplikasi sederhana atau simulasi, seperti sistem pemesanan tiket atau manajemen antrean, menggunakan ArrayList atau LinkedList.

PERSIAPAN

1. Siswa Muslim dapat membaca doa berikut sebelum memulai pelajaran:

رَضِيْتُ بِاللهِ رَبًّا وَبِالإِسْلَامِ دِيْنًا وَبِمُحَمَّدٍ نَبِيًّا وَرَسُوْلًا. رَبِّ زِدْنِيْ عِلْمًا وَارْزُقْنِيْ فَهْمًا

2. Laptop atau komputer pribadi
3. Java Development Kit
4. Java Runtime Environment
5. IDE (IntelliJ IDEA, Eclipse, NetBeans, dll.)
6. Motivasi dan komitmen yang kuat untuk belajar

KATA KUNCI

ArrayList, LinkedList, Node, Pointers, Head, Tail

1. Array List

1.1. Definisi ArrayList

ArrayList adalah struktur data dinamis di Java yang berfungsi seperti versi array yang lebih fleksibel. Tidak seperti array biasa yang memiliki ukuran tetap, ArrayList dapat secara otomatis berkembang atau menyusut tergantung pada jumlah elemen yang ditambahkan atau dihapus. ArrayList menyimpan data dalam urutan yang teratur dan memungkinkan akses cepat ke elemen mana pun menggunakan indeks. Java juga menyediakan banyak metode built-in untuk mengelola data, seperti menambah, menghapus, atau memperbarui elemen, membuat ArrayList lebih mudah digunakan daripada array tradisional. Secara internal, ArrayList diimplementasikan menggunakan array standar yang secara otomatis mengubah ukurannya sesuai kebutuhan, sehingga programmer tidak perlu mengelola memori secara manual.

ArrayList di Java menyediakan berbagai konstruktor untuk membuat ArrayList berdasarkan kebutuhan yang berbeda:

1) ArrayList()

Membuat ArrayList kosong dengan kapasitas awal default.

```
1  ArrayList<Integer> arr = new ArrayList<>();
```

2) ArrayList(Collection<? extends E> c)

Membuat ArrayList yang diinisialisasi dengan elemen dari koleksi yang ditentukan.

```
1  ArrayList<String> arr = new ArrayList<>(collection);
```

3) ArrayList(int initialCapacity)

Konstruktor ini digunakan untuk membangun array list dengan kapasitas awal yang telah ditentukan.

```
1  ArrayList<Double> arr = new ArrayList<>(20);
```

1.2. Karakteristik ArrayList

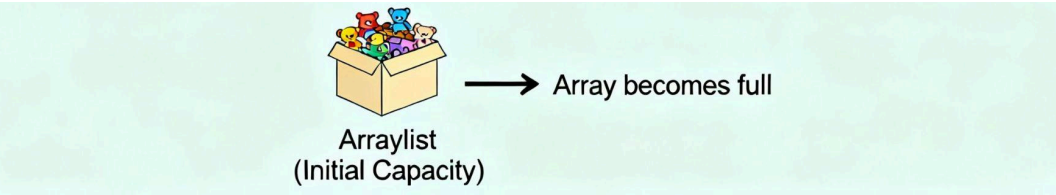
ArrayList memiliki beberapa karakteristik yang membedakannya dari struktur data lainnya. Pertama, semua elemen disimpan secara berurutan dan setiap elemen memiliki indeks yang jelas, sehingga kita dapat membaca atau mengubah data dengan sangat cepat. ArrayList juga memiliki banyak metode built-in seperti add, remove, search, dan size, sehingga pengguna tidak perlu membuat fungsi-fungsi tersebut secara manual.

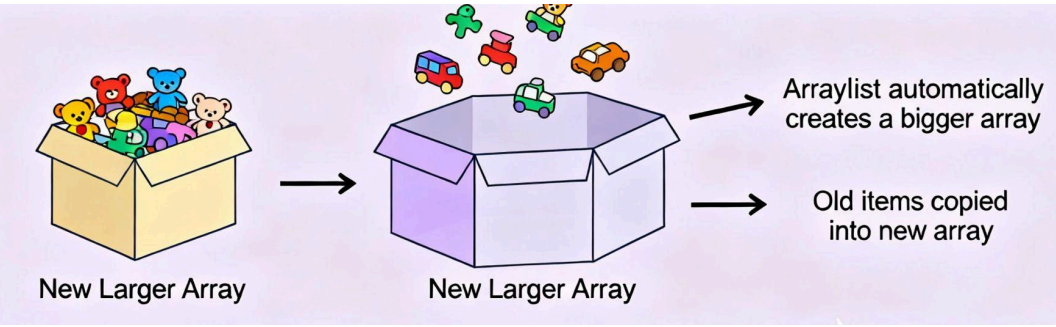
Karakteristik kunci lainnya adalah bahwa ArrayList mendukung generics, yang berarti kita dapat menentukan tipe data apa yang akan disimpannya (seperti String, Integer, dll.). Ini membantu mencegah kesalahan karena kita tidak dapat memasukkan tipe data yang tidak sesuai. ArrayList juga mengelola memori secara otomatis. Ketika lebih banyak elemen ditambahkan dan penyimpanan penuh, kapasitasnya akan meningkat secara otomatis. Pengguna juga dapat menetapkan kapasitas awal untuk menggunakan memori dengan lebih efisien.

Feature	Array	ArrayList
Size	Fixed size (cannot change)	Flexible size
Type of Data	Can store primitives and objects	Can only store objects (not primitives directly)
Built-in Methods	Very few (length only)	Many methods (add, remove, get, set, size, etc.
Generics Support	No	Yes
Memory Handling	Manual, fixed	Automatic resizing

1.3. Cara ArrayList Bekerja Secara Internal

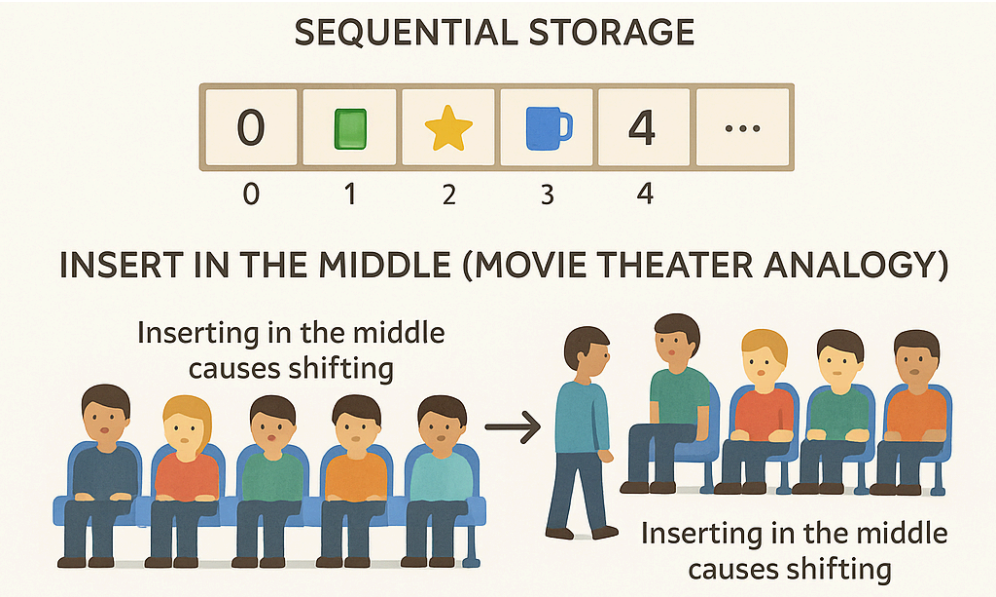
Di Java, ArrayList menyimpan datanya menggunakan array biasa. Namun, tidak seperti array biasa, ketika penyimpanan penuh, ArrayList tidak berhenti atau menampilkan pesan kesalahan. Sebaliknya, ArrayList secara otomatis membuat array yang lebih besar dan menyalin semua data lama ke dalamnya. Hal ini terjadi secara diam-diam di latar belakang, sehingga Anda dapat terus menambahkan item tanpa perlu khawatir tentang ukurannya.





Bayangkan sebuah kotak mainan. Anda terus menambahkan mainan hingga kotak penuh. Ketika penuh, Anda mendapatkan kotak yang lebih besar, memindahkan semua mainan ke dalamnya, lalu menambahkan mainan baru. Proses pemindahan ini membutuhkan waktu, tetapi tidak sering terjadi.

ArrayList juga menjaga semua elemen tetap berurutan. Jadi, jika Anda menyisipkan sesuatu di tengah, item lain harus bergeser untuk membuat ruang, seperti orang-orang yang bergerak sedikit di teater ketika seseorang ingin duduk di tengah. Itulah mengapa menyisipkan atau menghapus di tengah lebih lambat, sementara mengakses item berdasarkan indeks sangat cepat.



1.4. Metode ArrayList Penting

Untuk membantu Anda lebih memahami berbagai operasi yang dapat dilakukan pada ArrayList, tabel berikut merangkum metode yang paling sering digunakan. Ini juga mencakup penjelasan singkat dan analogi sederhana untuk membuat setiap metode lebih mudah dipahami.

Method	Function	Analogy
`add(element)`	Adds an element to the end	Placing a new book at the end of a bookshelf
`add(index, element)`	Inserts an element at a specific position	Slipping a book into the middle of a shelf, causing other books to shift
`get(index)`	Retrieves an element by position	Picking a book from the shelf based on its slot number
`set(index, element)`	Replaces an element at a specific position	Swapping an old book with a new one in the same spot
`remove(index)`	Removes an element at a specific position	Pulling a book from the middle of a shelf and other books move forward
`size()`	Returns the number of elements	Counting how many books are on the shelf

`isEmpty()`	Checks whether the list is empty	Checking whether the shelf is completely empty
`clear()`	Removes all elements	Taking all books off the shelf until it is completely empty

1.5. Contoh Kasus Penggunaan ArrayList

Studi Kasus: Dalam sistem pemesanan tiket kereta api, data penumpang harus dikelola dengan cepat dan rapi. Setiap kali penumpang memesan tempat duduk, sistem perlu menambahkan entri baru ke dalam daftar. Kadang-kadang ada penumpang prioritas (seperti lansia atau VIP) yang harus ditempatkan dalam urutan tertentu. Selain itu, penumpang dapat membatalkan tiket mereka dan admin perlu memperbarui data, memeriksa jumlah total penumpang yang telah memesan, atau menghapus daftar setelah jadwal keberangkatan selesai. Untuk menangani semua kebutuhan ini, ArrayList menyediakan berbagai metode yang sangat berguna untuk mengelola daftar penumpang kereta api. Berikut penjelasan setiap metode beserta contohnya dalam konteks pemesanan tiket kereta api.

1) **Deklarasi ArrayList**

Membuat ArrayList baru bernama buyers untuk menyimpan daftar pelanggan atau penumpang tiket kereta api. Penjelasan setiap bagian:

- **`ArrayList<String>`** berarti daftar ini hanya diizinkan untuk menyimpan data bertipe String. Dalam hal ini, setiap String berisi informasi penumpang seperti: "Andi - Gerbong 1 Kursi 12A"
- **`buyers`** adalah nama variabel. Ini merupakan daftar utama tempat semua data penumpang disimpan.
- **`new ArrayList<>()`** membuat objek ArrayList baru yang kosong.

```
1  ArrayList<String> buyers = new ArrayList<>();
```

```java
// Helper method to display the list of buyers
private static void displayList(ArrayList<String> buyers) {
    if (buyers.isEmpty()) {
        System.out.println("The buyer list is empty.");
    } else {
        System.out.println("Current Buyer List:");
        for (int i = 0; i < buyers.size(); i++) {
            System.out.println((i + 1) + ". " + buyers.get(i));
        }
    }
}
```

2) `add(element)`

Metode ini digunakan untuk menambahkan penumpang baru di akhir daftar pemesanan.

Contoh kasus: Ketika seseorang memesan tiket untuk gerbong dan kursi tertentu, data mereka ditambahkan sebagai entri baru.

```java
// 1. add(element) - Add new buyers
System.out.println("1. Adding new buyers using add(element)");
buyers.add("DIGGIE - Car 1 Seat 12A");
buyers.add("IXIA - Car 2 Seat 7C");
buyers.add("LING - Car 3 Seat 5B");
displayList(buyers);

```

3) `add(index, element)`

Metode ini menyisipkan penumpang pada posisi tertentu, lalu menggeser yang lain ke bawah. **Contoh kasus:** Penumpang senior atau VIP perlu ditempatkan di atas daftar pemrosesan untuk penanganan prioritas.

```java
// 2. add(index, element) - Insert priority buyers (senior/VIP)
System.out.println("\n2. Inserting a priority buyer at the first position using add(index, element)");
buyers.add(0, "Priority - XAVIER - Car 1 Seat 1A");
displayList(buyers);
```

4) `get(index)`

Digunakan untuk mendapatkan data penumpang pada posisi tertentu dalam daftar pemesanan.

Contoh kasus: Staf kereta perlu melihat detail penumpang ke-3 untuk keperluan verifikasi.

```
// 3. get(index) - Retrieve a specific buyer
        System.out.println("\n3. Retrieving the 3rd buyer using get(index)");
        String thirdBuyer = buyers.get(2);
        System.out.println("3rd Buyer: " + thirdBuyer);

```

5) `set(index,element)`

Mengganti informasi penumpang pada posisi tertentu tanpa mengubah ukuran daftar. **Contoh kasus:** Penumpang meminta untuk mengubah kursi (misalnya, pindah dari Gerbong 2 Kursi 7C ke Gerbong 2 Kursi 9D).

```
// 4. set(index, element) - Update buyer information
        System.out.println("\n4. Updating the 2nd buyer using set(index, element)");
        buyers.set(1, "AMON - Car 2 Seat 9D (Changed Seat)");
        displayList(buyers);
```

6) `remove(index)`

Menghapus penumpang pada posisi tertentu, dan semua penumpang di bawah posisi tersebut bergeser ke atas.

Contoh kasus: Penumpang membatalkan tiket kereta api mereka, sehingga entri mereka dihapus dari daftar pemesanan.

```
// 5. remove(index) - Remove a buyer who cancels
       System.out.println("\n5. Removing the 3rd buyer (ticket canceled) using remove(index)");
       buyers.remove(2);
       displayList(buyers);
```

7) `size()`

Mengembalikan jumlah penumpang yang saat ini tercantum dalam daftar pemesanan.

Contoh kasus: Untuk memeriksa berapa banyak penumpang yang telah memesan kursi sebelum keberangkatan.

```

// 6. size() - Count total buyers
System.out.println("\n6. Checking the total number of buyers using size()");
int total = buyers.size();
System.out.println("Total buyers who booked: " + total);
```

8) `isEmpty()`

Digunakan untuk memeriksa apakah daftar berisi penumpang.

Contoh kasus: Sebelum pemesanan dibuka, daftar akan kosong, jadi staf dapat memeriksa apakah sudah ada entri awal.

```
// 7. isEmpty() - Check if the list is empty
System.out.println("\n7. Checking if the buyer list is empty using isEmpty()");
if (buyers.isEmpty()) {
    System.out.println("No buyers have booked tickets yet.");
} else {
    System.out.println("The buyer list has entries.");
}
```

9) `clear()`

Menghapus semua data penumpang dari daftar pemesanan.

Contoh kasus: Setelah kereta berangkat, daftar pemesanan untuk perjalanan tersebut dihapus.

```
// 8. clear() - Clear the list after train departure
System.out.println("\n8. Clearing the list after the train has departed using clear()");
buyers.clear();
displayList(buyers);
System.out.println("isEmpty() after clear: " + buyers.isEmpty());
```

2. Linked List

2.1. Definisi LinkedList

LinkedList adalah struktur data yang terdiri dari serangkaian elemen yang terhubung melalui tautan. Tidak seperti ArrayList, yang menyimpan data dalam satu blok memori yang berkelanjutan, LinkedList menyimpan setiap elemen pada lokasi memori yang terpisah. Setiap elemen disebut node, dan setiap node memiliki dua bagian: data dan pointer yang menunjuk ke node berikutnya.

Anda dapat membayangkan LinkedList seperti kereta. Setiap gerbong (node) membawa penumpang (data) dan terhubung dengan gerbong berikutnya. Jika Anda ingin menambah atau menghapus gerbong, Anda hanya perlu mengubah koneksinya—Anda tidak perlu memindahkan gerbong lainnya. Ini membuat penambahan atau penghapusan elemen menjadi sangat cepat, terutama di awal atau di tengah daftar.

2.2. Struktur Dasar LinkedList

Sebelum memahami cara kerja LinkedList secara keseluruhan, penting untuk terlebih dahulu mempelajari komponen utama yang membentuk struktur ini.

1) **Node**

Node adalah elemen dasar yang membentuk LinkedList. Setiap node berisi dua bagian: data yang ingin Anda simpan dan pointer yang merujuk ke node lain. Dengan kata sederhana, node dapat berlokasi di mana saja di memori, tetapi pointer membuat semua node tetap terhubung.

Analogi: Node seperti gerbong kereta. Setiap gerbong memiliki konten (data) dan penghubung yang mengarahkannya ke gerbong berikutnya (pointer).



2) **Data**

Data adalah informasi utama yang disimpan pada node. Nilai ini dapat berupa angka, teks, objek, atau jenis data lainnya. Setiap node dapat menyimpan data yang berbeda, dan data tidak perlu disimpan di lokasi memori yang berdekatan.

Analogi: Data seperti penumpang atau kargo di dalam gerbong kereta.



3) **Next & Previous Point**

Next menunjuk ke node berikutnya dalam LinkedList. Pointer ini memungkinkan daftar untuk dilintasi dari awal hingga akhir. Jika pointer next bernilai null, berarti node tersebut merupakan node terakhir dalam daftar.

Analogi: Pointer next seperti penghubung depan gerbong kereta yang menghubungkannya dengan gerbong berikutnya.

Previous menunjuk ke node sebelumnya dan digunakan dalam Doubly Linked List. Dengan pointer ini, daftar dapat dilintasi maju dan mundur, sehingga navigasi menjadi lebih fleksibel.

Analogi: Pointer previous seperti penghubung belakang gerbong kereta memungkinkan gerakan dalam arah sebaliknya.



4) **Pointer**

Pointer adalah referensi yang menghubungkan satu node ke node berikutnya (atau sebelumnya). Pointer menciptakan rantai yang membentuk LinkedList, sehingga data dapat dilintasi.

Analogi: Pointer seperti penghubung antara gerbong kereta, menjaga semua gerbong tetap terhubung bahkan ketika tidak diletakkan rapat.



5) **Head**

Head adalah node pertama dalam linked list. Semua operasi, seperti lintasan, pencarian, atau penghitungan node, selalu dimulai dari head.

Analogi: Head seperti gerbong depan kereta, di mana Anda mulai memeriksa seluruh kereta dari depan.



6) **Tail**

Tail adalah node terakhir dalam LinkedList. Dalam daftar (linear) normal, pointer next pada tail adalah null. Namun, dalam Circular LinkedList, tail terhubung kembali ke head, sehingga membentuk loop.

Analogi: Tail seperti gerbong terakhir kereta, dan dalam versi circular, gerbong terakhir ini terhubung kembali ke gerbong depan.



2.3. Jenis-Jenis LinkedList

LinkedList memiliki beberapa variasi, masing-masing dirancang dengan struktur pointer dan perilaku yang berbeda. Meskipun semuanya menyimpan data dalam bentuk node yang terhubung, cara setiap node terhubung ke node lainnya menentukan cara kerja daftar tersebut. Memahami jenis-jenis ini membantu developer memilih LinkedList yang tepat untuk penyisipan, penghapusan, pencarian, atau manajemen data sirkular.

1) **Singly Linked List**

Singly Linked List adalah jenis LinkedList yang paling sederhana. Setiap node hanya memiliki satu pointer dan pointer itu menunjuk ke node berikutnya. Ini memungkinkan daftar dibaca dari node pertama ke node terakhir dalam satu arah.

Karena hanya bergerak maju, operasi seperti menyisipkan atau menghapus node di awal daftar sangat cepat. Namun, Singly Linked List tidak dapat mundur karena tidak memiliki pointer ke node sebelumnya.



2) **Doubly Linked List**

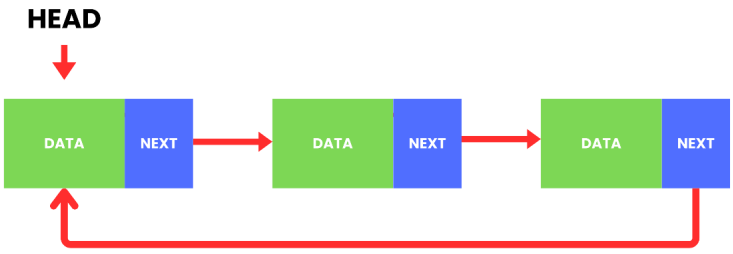
Doubly Linked List adalah versi LinkedList yang lebih fleksibel karena setiap node memiliki dua pointer: satu menunjuk ke node sebelumnya dan satu menunjuk ke node berikutnya. Dua arah ini memungkinkan daftar dilintasi maju dan mundur, sehingga pencarian atau penghapusan elemen tertentu menjadi lebih mudah.

Namun, kemampuan dua arah ini memerlukan lebih banyak memori, karena setiap node harus menyimpan pointer tambahan. Doubly Linked List sangat berguna ketika aplikasi membutuhkan navigasi dua arah, seperti dalam navigasi playlist, sistem undo-redo, atau proses apa pun yang memerlukan pergerakan bolak-balik melalui data.

3) **Singly Circular Linked List**

Singly Circular Linked List memiliki struktur yang mirip dengan Singly Linked List, tetapi node terakhir tidak menunjuk ke node null. Sebaliknya, node terakhir menunjuk kembali ke node pertama, sehingga membentuk lingkaran. Karena struktur sirkular ini, daftar dapat dilintasi secara terus-menerus tanpa mencapai akhir, sehingga berguna untuk sistem yang memerlukan loop tak terbatas, seperti antrean atau rotasi giliran.

Meskipun lintasannya masih searah, sifat circular memastikan bahwa daftar selalu dapat kembali ke awal tanpa perlu memeriksa bagian akhir.



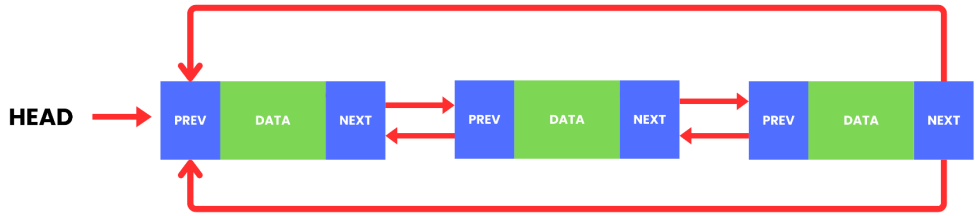
4) **Doubly Circular Linked List**

Doubly Circular Linked List adalah bentuk paling lengkap dari semua jenis LinkedList. Setiap node memiliki dua pointer - satu ke node sebelumnya dan satu ke node berikutnya. Namun, tidak seperti Doubly Linked List biasa, node terakhir dalam struktur ini menunjuk kembali ke node pertama, dan node pertama menunjuk kembali ke node terakhir, sehingga membentuk struktur circular dua arah.

Struktur circular dua arah ini memungkinkan gerakan maju dan mundur tanpa batas, menjadikannya ideal untuk aplikasi yang memerlukan navigasi circular berkelanjutan, seperti carousel menu, rotasi pemain dalam permainan, atau simulasi

rotasi antrean.

Namun, jenis ini menggunakan paling banyak memori, karena setiap node menyimpan dua pointer dan daftar membentuk koneksi sirkular.



2.4. Operasi LinkedList Penting

LinkedList menyediakan beberapa operasi penting yang memungkinkan kita menambah, menghapus, dan melintasi data secara fleksibel.

1) Insertion

Insertion adalah operasi untuk menambahkan node baru ke dalam linked list. Node dapat ditambahkan di awal, di tengah, atau di akhir daftar. Dalam LinkedList, insertion sangat efisien karena hanya memerlukan pembaruan pointer.

1. **Insert di Head:** Tambahkan node baru, arahkan ke head lama, lalu jadikan node baru sebagai head baru.
2. **Insert di Tengah:** Arahkan node baru ke node berikutnya, lalu perbarui pointer node sebelumnya agar terhubung ke node baru.
3. **Insert di Tail:** Arahkan node terakhir ke node baru, lalu tetapkan node baru sebagai tail baru.

Analogi: Menambahkan gerbong kereta baru sangat mudah—cukup dipasang di depan, di tengah, atau di belakang tanpa memindahkan gerbong lainnya.

2) Deletion

Deletion adalah operasi untuk menghapus node dari linked list. Ini dapat dilakukan di awal, tengah, atau akhir daftar dengan menyesuaikan pointer agar mereka melewati node yang telah dihapus.

- **Delete Head:** Pindahkan head ke node berikutnya.
- **Delete Middle:** Hubungkan node sebelumnya langsung ke node berikutnya setelah node yang dihapus.
- **Delete Tail:** Perbarui pointer node sebelum tail sehingga menunjuk ke null.

Analogi: Menghapus gerbong kereta sangat sederhana—cukup lepaskan dan hubungkan kembali gerbong di sekitarnya, tanpa membangun ulang seluruh kereta.

3) Traversal

Traversal adalah proses mengunjungi setiap node dalam LinkedList, dimulai dari head dan berlanjut hingga node terakhir. Karena LinkedList tidak mendukung akses langsung (random), traversal harus dilakukan pada satu node pada satu waktu dengan mengikuti pointer next.

Analogi: Untuk memeriksa semua gerbong kereta dari depan ke belakang, Anda harus berjalan dari gerbong pertama ke gerbong berikutnya, lalu terus berurutan hingga gerbong terakhir.

4) Searching

Searching dilakukan dengan melintasi LinkedList dari head hingga Anda menemukan node yang berisi nilai yang diinginkan. Karena LinkedList tidak memiliki indeks seperti ArrayList, pencarian harus dilakukan secara linear, dari awal hingga akhir.

Analog: Mencari penumpang tertentu di kereta berarti Anda harus memeriksa setiap gerbong satu per satu sampai Anda menemukannya.

5) Updating

Updating digunakan untuk mengganti data yang disimpan dalam node tanpa mengubah struktur LinkedList. Anda pertama-tama menggunakan lintasan untuk menemukan node yang ingin Anda perbarui, lalu cukup ubah nilainya.

Analogi: Jika Anda ingin memperbarui daftar penumpang di gerbong kereta tertentu, Anda cukup membuka gerbong tersebut dan mengubah informasinya. Tidak perlu memindahkan atau menyambung ulang gerbong apa pun.

6) Menghitung Node

Menghitung jumlah node dilakukan dengan menelusuri dari head ke akhir sambil menambah counter untuk setiap node yang dikunjungi.

Analogi: Untuk mengetahui berapa banyak gerbong kereta yang ada, Anda perlu berjalan melalui seluruh kereta, bergerak dari satu gerbong ke gerbong berikutnya dan menghitungnya satu per satu.

2.5. Contoh Kasus Penggunaan LinkedList

Studi Kasus: Mengelola Antrean Penumpang Kereta dengan LinkedList. Dalam sistem tiketisasi dan boarding kereta api, data penumpang sering berubah secara dinamis. Beberapa penumpang tiba lebih awal, beberapa masuk dengan prioritas khusus, beberapa membatalkan perjalanan mereka, dan kadang-kadang staf perlu menyusun ulang urutan penumpang untuk pemeriksaan tiket.

Class `Node.Java`

```java
class Node {
    String name;
    Node next;

    public Node(String name) {
        this.name = name;
        this.next = null;
    }
}
```

Class `TrainPassengerList.Java`

```java
class TrainPassengerList {

    Node head;
    Node tail;
```

1. `addPriorityPassenger`

```java
// Insert priority passenger (insert at head)
public void addPriorityPassenger(String name) {
    Node newNode = new Node(name);
    newNode.next = head;
    head = newNode;

    if (tail == null) {
        tail = newNode;
    }
}
```

2. addNormalPassenger

```java
// Insert normal passenger (insert at tail)
 public void addNormalPassenger(String name) {
     Node newNode = new Node(name);

     if (head == null) {
         head = newNode;
         tail = newNode;
         return;
     }

     tail.next = newNode;
     tail = newNode;
 }
```

3. addAfter

```java
// Insert after a certain passenger
    public void addAfter(String keyPassenger, String newPassenger) {
        Node current = head;

        while (current != null && !current.name.equals(keyPassenger)) {
            current = current.next;
        }

        if (current != null) {
            Node newNode = new Node(newPassenger);
            newNode.next = current.next;
            current.next = newNode;

            if (current == tail) {
                tail = newNode;
            }
        }
    }

```

4. addAfter

```java
// Remove passenger who cancels (delete by name)
public void removePassenger(String name) {
    if (head == null) return;

    if (head.name.equals(name)) {
        head = head.next;
        if (head == null) tail = null;
        return;
    }

    Node current = head;
    Node previous = null;

    while (current != null && !current.name.equals(name)) {
        previous = current;
        current = current.next;
    }

    if (current != null) {
        previous.next = current.next;
        if (current == tail) {
            tail = previous;
        }
    }
}
```

5. displayQueue

```java
// Traverse list
public void displayQueue() {
    Node current = head;
    System.out.println("Current Passenger Queue:");
    while (current != null) {
        System.out.println("- " + current.name);
        current = current.next;
    }
}
```

6. searchPassenger

```java
// Search passenger
public boolean searchPassenger(String name) {
    Node current = head;
    while (current != null) {
        if (current.name.equals(name)) return true;
        current = current.next;
    }
    return false;
}
```

7. updatePassenger

```java
// Update passenger data
public void updatePassenger(String oldName, String newName) {
    Node current = head;
    while (current != null) {
        if (current.name.equals(oldName)) {
            current.name = newName;
            return;
        }
        current = current.next;
    }
}
```

8. `countPassengers`

```java
// Count passenger
public int countPassengers() {
    int count = 0;
    Node current = head;
    while (current != null) {
        count++;
        current = current.next;
    }
    return count;
}
```

9. Class `TrainPassenger.java`

```java
private TrainPassengerList passengerList;

    public TrainPassenger() {
        this.passengerList = new TrainPassengerList();
    }

    // Delegating methods to the TrainPassengerList
    public void addNormalPassenger(String name) {
        passengerList.addNormalPassenger(name);
    }

    public void addPriorityPassenger(String name) {
        passengerList.addPriorityPassenger(name);
    }

    public void displayQueue() {
        passengerList.displayQueue();
    }

    public void addAfter(String keyPassenger, String newPassenger) {
        passengerList.addAfter(keyPassenger, newPassenger);
    }

    public void removePassenger(String name) {
        passengerList.removePassenger(name);
    }

    public boolean searchPassenger(String name) {
        return passengerList.searchPassenger(name);
    }

    public void updatePassenger(String oldName, String newName) {
        passengerList.updatePassenger(oldName, newName);
    }

    public int countPassengers() {
        return passengerList.countPassengers();
    }

```

```java
public static void main(String[] args) {

        TrainPassenger queue = new TrainPassenger();

        System.out.println("=== TRAIN PASSENGER QUEUE SIMULATION USING LINKEDLIST ===\n");

        System.out.println("Adding normal passengers...");
        queue.addNormalPassenger("GRANGER");
        queue.addNormalPassenger("OBSIDIA");
        queue.addNormalPassenger("BRUNO");
        queue.displayQueue();

        System.out.println("\nAdding PRIORITY passenger: 'MIYA'...");
        queue.addPriorityPassenger("MIYA");
        queue.displayQueue();

        System.out.println("\nInserting 'Family Group' after 'MIYA'...");
        queue.addAfter("MIYA", "Family Group");
        queue.displayQueue();

        System.out.println("\nRemoving passenger 'OBSIDIA' (canceled)...");
        queue.removePassenger("OBSIDIA");
        queue.displayQueue();

        System.out.println("\nSearching for passenger 'BRUNO'...");
        System.out.println(queue.searchPassenger("BRUNO") ? "Found" : "Not Found");

        System.out.println("\nUpdating passenger 'Family Group'...");
        queue.updatePassenger("Family Group", "Family Group (Updated)");
        queue.displayQueue();

        System.out.println("\nTotal passengers in queue: " + queue.countPassengers());
    }
```

CODELAB

Lengkapi kode berikut sehingga berjalan dengan benar.

Array And Linked

```java
package Codelab;
import java.util.ArrayList;
import java.util.LinkedList;

public class ArrayAndLinked {

    public static void main(String[] args) {
        // 1) ArrayList declaration
        ArrayList<String> tickets = new __________<>();

        // 2) add(element) - Add a ticket to the end of the list
        tickets.__________("T001");
        tickets.__________("T002");
        tickets.__________("T003");
        tickets.__________("T004");

        System.out.println("=== TRAIN TICKET ARRAYLIST ===");
        System.out.println("Ticket list: " + tickets);

        // 3) get(index) - Get a ticket based on its position
        System.out.println("First ticket: " + tickets.__________(0));

        // 4) add(index, element) - Insert a priority ticket at a specific position
        tickets.__________(0, "T000-PRIORITY");
        System.out.println("After adding priority ticket: " + tickets);

        // 5) set(index, element) - Update a ticket at a specific position
        tickets.__________(1, "T001-UPDATED");
        System.out.println("After updating T001: " + tickets);

        // 6) remove(element) - Remove a specific ticket
        tickets.__________("T002");
        System.out.println("After removing T002: " + tickets);

        // 7) size() - Check the number of tickets
        System.out.println("Number of tickets: " + tickets.__________());

        // 8) isEmpty() - Check whether the list is empty
        System.out.println("Is the list empty? " + tickets.__________());

        // 9) clear() - Remove all tickets
        tickets.__________();
        System.out.println("After clear: " + tickets);
        System.out.println("Is the list empty after clear? " + tickets.__________());
```

Array And Linked

```
// 1) LinkedList declaration
        LinkedList<String> queue = new ___________<>();

        // 2) add(element) - Add a ticket to the end of the queue
        queue.___________("GA031");
        queue.___________("GA037");
        queue.___________("GA042");

        System.out.println("\n=== AIRPLANE TICKET LINKEDLIST QUEUE ===");
        System.out.println("Initial queue: " + queue);

        // 3) addFirst() - Add a priority ticket to the beginning of the queue (HEAD)
        queue.___________("GA045-PRIORITY");
        System.out.println("After adding priority ticket at the beginning: " + queue);

        // 4) addLast() - Add a new ticket to the end of the queue (TAIL)
        queue.___________("GA048");
        System.out.println("After adding a ticket at the end: " + queue);

        // 5) getFirst() - View the first ticket (HEAD)
        System.out.println("First ticket (HEAD): " + queue.___________());

        // 6) getLast() - View the last ticket (TAIL)
        System.out.println("Last ticket (TAIL): " + queue.___________());

        // 7) removeFirst() - Remove the first ticket after it is served
        queue.___________();
        System.out.println("After serving the first customer: " + queue);

        // 8) size() - Check the number of tickets in the queue
        System.out.println("Number of tickets in queue: " + queue.___________());
    }
}
```

DEMO

TASK 1 Passenger Management System (ArrayList dan LinkedList)

Sepanjang praktikum ini, Anda akan secara bertahap mengembangkan Sistem Pemesanan Makanan. Sistem akan terus berkembang pada setiap modul seiring dengan mempelajari dan menerapkan konsep struktur data baru. Dalam modul ini, Anda akan memperluas sistem untuk menangani pesanan pelanggan ganda dan item ganda per pesanan menggunakan struktur data Java List (ArrayList dan LinkedList).

Siswa harus mengimplementasikan persyaratan berikut:

1. Buat kelas bernama OrderItem untuk menyimpan item menu individual, termasuk nama menu, kategori makanan (FoodCategory), dan jumlah.
2. Perbarui kelas FoodOrder agar dapat menyimpan beberapa objek OrderItem menggunakan LinkedList<OrderItem>. Implementasikan metode addItem(OrderItem item) untuk menyisipkan item ke dalam daftar.
3. Buat kelas bernama OrderManager untuk mengelola semua pesanan pelanggan aktif menggunakan ArrayList<FoodOrder>.
4. Implementasikan metode dalam OrderManager untuk menambahkan pesanan, menampilkan semua pesanan, serta mencari dan menghapus pesanan berdasarkan kode pesanan.
5. Dalam kelas Main, implementasikan loop bersarang menggunakan Scanner untuk memungkinkan pengguna memasukkan beberapa pesanan pelanggan (ArrayList) dengan beberapa item menu per pesanan (LinkedList).
6. Output sesuai persyaratan.

Output:

```
=== Food Ordering System ===

--- Entering Customer Order ---
Enter Customer Name: Ibrahim
Enter Customer ID: 12345678
Enter Order Code: ODR-01
Enter Menu Name: Ayam Krispi Bakar
Select Food Category:
1. FOOD
2. BEVERAGE
3. DESSERT
Enter choice: 1
Enter Quantity: 1
Add another item to this order? (y/n): y
Enter Menu Name: Es Teh
Select Food Category:
1. FOOD
2. BEVERAGE
3. DESSERT
Enter choice: 2
Enter Quantity: 1
Add another item to this order? (y/n): n

Add another customer order? (y/n): y

--- Entering Customer Order ---
Enter Customer Name: Nero
Enter Customer ID: Aschente
Enter Order Code: ODR-02
Enter Menu Name: Ayam Goreng Bakar
Select Food Category:
1. FOOD
2. BEVERAGE
3. DESSERT
Enter choice: 1
Enter Quantity: 1
Add another item to this order? (y/n): n

Add another customer order? (y/n): n
```

```
================================
=== ALL ACTIVE ORDERS ===

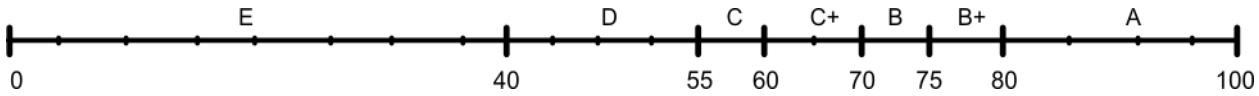
================================
Order Code       : ODR-01
Customer Name    : Ibrahim
Customer ID      : 12345678
Customer ID Type : Long
Ordered Items:
  - Ayam Krispi Bakar x1 (FOOD)
  - Es Teh x1 (BEVERAGE)

================================
Order Code       : ODR-02
Customer Name    : Nero
Customer ID      : Aschente
Customer ID Type : String
Ordered Items:
  - Ayam Goreng Bakar x1 (FOOD)
```

Grading Rubric

Assessment Aspect	Points (Total 100%)
Codelab	**Total 10%**
Successfully run	20%
Demo	**Total 80%**
Implement Arraylist	20%
Implement LinkedList	20%
Running Smoothly	10%
Answer Questions	30%

Grading Scale



A = (81 - 100) → GOAT

B+ = (75 - 80) → Very Good

B = (70 - 74) → Good

C+ = (60 - 69) → Fairly Good

C = (55 - 59) → Fair

D = (41 - 54) → Poor

E = (0 - 40) → Bro really...